What Makes Traces Work for Verdict Tasks? An Empirical Decomposition of Execution Trace SFT

Anonymous

Abstract

Supervised fine-tuning (SFT) on execution traces has shown promise for improving code understanding, yet the mechanisms underlying its effectiveness remain poorly understood. We study trace SFT for *verdict tasks*—binary pass/fail predictions over test outcomes—and find that a compact, verdict-optimized trace abstraction (VOTA) achieves 99.2% accuracy, dramatically outperforming both no-trace (75.1%) and full-trace (81.8%) baselines. Through systematic component ablation, however, we discover that VOTA's near-perfect performance is almost entirely attributable to a single signal: the presence or absence of runtime exceptions. An exception-only condition achieves 98.0%, statistically indistinguishable from full VOTA ($p = 0.18$), while a training-free rule matching `ERROR` keywords reaches 93.0%. Loop compression and assertion snapshots contribute negligibly beyond exception information. These findings replicate across two model families (CodeLlama-7B, DeepSeek-Coder-6.7B) but are *task-dependent*: on a bug detection verdict task where failures manifest as logic errors rather than runtime exceptions, VOTA's non-exception components become significant ($p < 0.01$). We conclude that for exception-dominated verdict tasks, trace SFT effectiveness reduces to error signal extraction rather than execution-grounded reasoning, and we argue that trace design should be guided by task-specific information structure analysis rather than universal compression principles.

1 Introduction

Execution traces—the runtime logs produced when code is executed against test cases—contain rich information about program behavior. Recent work has explored supervised fine-tuning (SFT) on execution traces to improve code language models Ni et al. (2024); Jain et al. (2024); Shi et al. (2024), with studies examining various trace formats on generative tasks such as code generation and repair. However, no prior work has systematically studied trace SFT for *verdict tasks*: binary classification problems where the model must predict a discrete outcome (e.g., test pass/fail) from code and optional trace information.

Verdict tasks occupy a distinct point in the task complexity spectrum. Unlike generation tasks that require producing multi-token outputs, verdict tasks have $O(1)$ output entropy—a single bit of information. This raises a fundamental question: *does trace granularity need to match task complexity?* If verdict tasks need only a binary signal, perhaps full execution traces provide too much information, diluting the decision-relevant signal among irrelevant execution details.

We introduce VOTA (Verdict-Optimized Trace Abstraction), a compact trace representation that retains only three components hypothesized to be decision-relevant: exception summaries, assertion-point snapshots (actual vs. expected values), and loop compression summaries. On test verdict prediction, VOTA achieves 99.2% accuracy, a +24.0 percentage point (pp) improvement over no-trace baselines and +17.4pp over full-trace SFT.

This headline result, however, tells only half the story. Through systematic ablation, we decompose VOTA's effectiveness and arrive at a sobering finding: **exception summaries alone account for nearly all of VOTA's performance**. An exception-only condition (98.0%) is statistically indistinguishable from full VOTA ($p = 0.18$, McNemar's test), while loop compression adds nothing beyond no-trace baselines ($p = 0.72$). A training-free rule—*if trace contains* `ERROR` *then* `fail`—achieves 93.0%, approaching the SFT models without any learned parameters.

These results force us to reframe what trace SFT accomplishes for verdict tasks. Rather than learning execution-grounded reasoning, the model primarily learns to extract and act on error presence signals—a near-trivial pattern that nonetheless requires SFT to handle the remaining 7% of ambiguous cases where error signals are absent or misleading. Crucially, this finding is *task-dependent*: on DebugBench bug detection, where failures arise from logic errors without runtime exceptions, exception-only drops to 81.5% while full VOTA achieves 88.5% ($p < 0.01$), demonstrating that VOTA's richer components become necessary when the error signal alone is insufficient.

Our contributions are:

1. An empirical decomposition showing that VOTA's verdict prediction effectiveness is dominated by error signal extraction, not multi-component behavioral understanding (§4.2).
2. Evidence that trace information *density*, not quantity, determines SFT effectiveness: full traces underperform VOTA by 17.4pp, with 74% of this gap attributable to information overload and 26% to gradient dilution (§4.4).
3. A cross-task analysis revealing that the dominance of error signals is task-dependent, with non-exception trace components becoming significant for logic-error verdict tasks (§4.6).

2 Related Work

Trace SFT for Code Models. DCSH Ni et al. (2024) evaluated five execution trace formats for SFT on generative code tasks, finding limited benefits. TracePile Shi et al. (2024) scaled trace SFT to 2.6M samples but evaluated only generation tasks. TRACED Jain et al. (2024) demonstrated that execution traces improve encoder-based code classification, but did not study decoder SFT for verdict tasks. Our work complements these studies by systematically examining trace SFT on classification (verdict) tasks and decomposing the mechanisms of effectiveness.

Information Density in Language Model Training. Prior work has shown that training data quality and information density affect SFT outcomes Zhou et al. (2024). Our trace length ablation (§4.4) provides direct evidence that higher information density per token—achieved through targeted trace compression—outperforms longer but noisier traces, even when the longer traces contain all the same decision-relevant signals.

Test Outcome Prediction. Predicting test outcomes from code has been studied in software engineering Chen et al. (2022), but typically using hand-crafted features rather than LLM-based approaches. Our work frames test verdict prediction as an SFT task and uses it as a lens to study trace information structure.

3 Method

3.1 Task Definition

Given source code c, a test function t, and an optional execution trace τ, the verdict task requires predicting a binary label $y \in \{\texttt{pass}, \texttt{fail}\}$ indicating whether executing t against c produces a passing or failing outcome.

3.2 VOTA: Verdict-Optimized Trace Abstraction

VOTA compresses full execution traces into three components:

Condition	Acc.	Δ no_tr.	Pass F1	Fail F1
VOTA	99.2	+24.0	.994	.986
exception_only	98.0	+22.9	.987	.960
full_trace	81.8	+6.6	—	—
no_trace	75.1	—	—	—

Table 1: Test verdict prediction accuracy (%) on CodeLlama-7B ($n = 1{,}190$). VOTA achieves near-perfect accuracy, but exception_only is statistically indistinguishable from VOTA (McNemar $p = 0.18$).

1. **Exception Summary**: A structured representation of runtime errors, formatted as `[ERROR] Traceback...` for failures or `[No exceptions]` for clean execution. This captures whether the execution terminated abnormally.
2. **Assertion Snapshots**: Actual vs. expected value pairs at assertion points, capturing the concrete values that determine test assertions (e.g., `actual=42, expected=43`).
3. **Loop Compression**: Iteration counts and convergence summaries for loops, abstracting repetitive execution patterns into compact descriptions.

The resulting VOTA traces average ∼80 tokens, compared to ∼3,200 tokens for full traces—a 40× compression ratio.

3.3 Training Setup

We fine-tune CodeLlama-7B Rozière et al. (2023) and DeepSeek-Coder-6.7B Guo et al. (2024) using standard SFT with cross-entropy loss on the verdict token. All models are trained for 3 epochs with learning rate 2×10^{-5}, batch size 16, and evaluated on a held-out test set of $n = 1{,}190$ samples (majority baseline: 71.5% pass).

3.4 Experimental Conditions

We compare the following conditions on the primary task (test verdict prediction):

- **no_trace**: Code + test only, no trace information.
- **full_trace**: Code + test + complete execution trace.
- **VOTA**: Code + test + verdict-optimized trace abstraction.
- **exception_only**: VOTA with only the exception summary component.
- **ae_only**: VOTA with only assertion snapshots (actual/expected values).
- **loop_only**: VOTA with only loop compression summaries.
- **trivial_rule**: A training-free baseline: `ERROR` in trace → fail; `ASSERT` without `ERROR` → pass; no trace → majority vote.

4 Experiments

4.1 Main Results: The Performance Gradient

Table 1 presents the primary results on test verdict prediction with CodeLlama-7B.

Condition	Acc. (%)	Δ no_tr.	p vs. exc.
VOTA (full)	99.2	+24.0	—
exception_only	98.0	+22.9	—
ae_only†	87.5	+12.4	<0.001
loop_only	75.5	+0.4	<0.001
no_trace	75.1	—	<0.001

Table 2: Component ablation ($n = 1{,}190$). †ae_only results have fallback contamination (see §4.2.1). p-values are McNemar's test vs. exception_only.

The performance gradient no_trace (75.1%) < full_trace (81.8%) < VOTA (99.2%) confirms that (a) traces help verdict prediction, and (b) compact, decision-relevant traces dramatically outperform verbose full traces. However, the exception_only condition (98.0%) achieves performance within 1.2pp of VOTA, with the gap not reaching statistical significance (McNemar $p = 0.18$, 95% CI of gap: $[-0.5, +2.9]$pp).

4.2 Component Ablation: Decomposing VOTA

Table 2 presents the full component ablation.

The results reveal a stark hierarchy:

- **Exception summaries dominate**: exception_only captures 95.4% of VOTA's lift over no_trace (22.9/24.0pp).
- **Loop compression contributes nothing**: loop_only (75.5%) is statistically indistinguishable from no_trace (75.1%), with $p = 0.72$. The 95% CI for lift includes zero $[-2.5, +3.3]$pp.
- **Assertion snapshots provide moderate signal**: ae_only achieves 87.5% (+12.4pp), but this is confounded by fallback contamination (§4.2.1).

We find no evidence that assertion snapshots or loop compression contribute meaningfully beyond what exception summaries alone provide for test verdict prediction.

4.2.1 ae_only Caveat

The ae_only condition's 87.5% accuracy should be interpreted cautiously. A subset of samples used fallback templates due to trace generation failures, introducing systematic bias. Moreover, prior ablation showed that removing assertion snapshots from VOTA does not decrease accuracy (99.4% without ae vs. 99.2% with ae), confirming that assertion information is redundant in the presence of exception summaries. The ae_only signal likely derives from actual/expected mismatches serving as a noisy proxy for error presence, rather than an independent reasoning channel.

4.3 The Trivial Rule Baseline

To further probe whether VOTA's effectiveness reflects genuine reasoning, we evaluate a training-free rule-based baseline (Table 3).

The trivial rule achieves 93.0% (Pass F1: 0.952, Fail F1: 0.872) by simply checking for `ERROR`/`Exception` keywords, demonstrating that the dominant signal in verdict traces is error presence—a surface-level pattern accessible without model training. SFT's marginal contribution over the trivial rule is ∼5–6pp, likely from handling: (1) edge cases where error signals are absent but the test fails (e.g., wrong output without exceptions), (2) flaky tests where errors are present but the test passes, and (3) samples without trace information.

Condition	Acc. (%)	Training?	GPU-hrs
VOTA SFT	99.2	Yes	2
exception_only SFT	98.0	Yes	2
trivial_rule	93.0	No	0
no_trace SFT	75.1	Yes	1

Table 3: A training-free rule (`has_ERROR` $\rightarrow$ `fail`) achieves 93.0% accuracy, capturing the majority of VOTA's signal without any learned parameters ($n = 1{,}190$).

Trace Length	Acc. (%)	Tokens/sample
VOTA ($\sim$80 avg)	99.2	80
256 tokens	86.2	256
512 tokens	84.5	512
1024 tokens	83.0	1,024
2048 tokens	82.0	2,048
Full ($\sim$3200 avg)	81.8	3,200
No trace	75.1	0

Table 4: Accuracy vs. trace length ($n = 1{,}190$). Longer traces do not improve accuracy; performance *decreases* monotonically from 256 to full length, consistent with signal dilution. VOTA at $\sim$80 tokens outperforms all truncation lengths.

4.4 Trace Length and Information Overload

Table 4 shows accuracy as a function of trace length for truncated full traces.

A counter-intuitive pattern emerges: accuracy *decreases* as trace length increases from 256 to full ($\sim$3,200) tokens. All truncated traces contain more raw information than VOTA's 80-token summaries, yet VOTA outperforms the best truncation (256 tokens) by 13.0pp. This demonstrates that the full trace *contains* the decision-relevant signal (ERROR presence) but buries it among irrelevant execution details, reducing the model's ability to extract it reliably.

The monotonic decrease from 256 $\rightarrow$ 3,200 tokens is consistent with signal dilution: as more noise tokens are added, the signal-to-noise ratio decreases and the model's pattern-matching accuracy degrades. Prior causal analysis attributed $\sim$26% of the full-trace–VOTA gap to gradient dilution (addressed by label-only loss masking, which improved full-trace to 86.2%) and $\sim$74% to information overload that only trace compression can resolve.

4.5 Cross-Model Replication

We replicate the core conditions on DeepSeek-Coder-6.7B (Table 5).

All key findings replicate: (1) the gradient no_trace $<$ full_trace $<$ VOTA holds; (2) exception_only $\approx$ VOTA on DeepSeek-Coder (gap 0.6pp, $p = 0.45$); (3) full_trace is substantially below VOTA despite containing the same error signals. The cross-model consistency confirms that these results reflect task information structure rather than model-specific reasoning capabilities.

4.6 Cross-Task Generalization: Bug Detection

To test whether error signal dominance is universal across verdict tasks, we evaluate on DebugBench bug detection (Python subset, $n = 800$), where the task is to predict whether code contains a bug (Table 6).

On DebugBench, VOTA achieves 88.5% (+19.5pp over no-trace baseline of 69.0%), confirming trace SFT value. However, exception_only achieves only 81.5%, with a significant 7.0pp gap from VOTA ($p < 0.01$). This gap arises because logic bugs—the dominant failure mode in DebugBench—produce incorrect outputs without triggering runtime exceptions. For such bugs, assertion snapshots

Model	no_tr.	full_tr.	VOTA	exc._only
CodeLlama-7B	75.1	81.8	99.2	98.0
DeepSeek-Coder	75.6	80.5	97.8	97.2

Table 5: Cross-model replication (n = 1,190). The performance gradient and exception-only equivalence replicate on DeepSeek-Coder-6.7B (VOTA vs. exception_only gap: 0.6pp, $p = 0.45$).

Task	no_tr.	exc._only	VOTA	exc.–VOTA gap
Test Verdict	75.1	98.0	99.2	-1.2 (n.s.)
Bug Detection	69.0	81.5	88.5	-7.0 ($p<0.01$)

Table 6: Cross-task comparison. On test verdict prediction, exception_only $\approx$ VOTA. On bug detection, the gap is significant ($p < 0.01$), as logic bugs do not trigger runtime exceptions.

(actual vs. expected value mismatches) and execution flow information provide decision-relevant signals that exception summaries cannot capture.

This cross-task comparison yields a key insight: *the dominance of error signals is task-dependent*. For verdict tasks where failures manifest as runtime exceptions (test outcomes), error flag extraction suffices. For verdict tasks involving logic errors (bug detection), richer trace components become necessary.

4.7 Calibration Analysis

Table 7 reports Expected Calibration Error (ECE) and Brier scores across conditions.

VOTA and exception_only show excellent calibration (ECE $\leq$ 0.015), though this is largely a consequence of near-perfect accuracy: when the signal is strong and unambiguous, the model produces high-confidence predictions that are almost always correct. The more informative comparison is between full_trace (ECE = 0.085) and no_trace (ECE = 0.165): full traces introduce sufficient information to reduce uncertainty relative to no-trace, but the noise in full traces prevents the model from achieving the calibration quality of the compact representations.

4.8 Seed Robustness

Table 8 confirms VOTA's stability across random seeds.

The extremely low cross-seed variance (std = 0.2pp) is consistent with a strong, easily-captured signal: any initialization quickly learns the ERROR $\rightarrow$ fail mapping in early training. While this demonstrates reliability, it also corroborates that the learned decision function is dominated by a surface-level pattern rather than a complex reasoning strategy that might be sensitive to initialization.

5 Discussion

What does trace SFT actually learn for verdict tasks? Our decomposition reveals that for test verdict prediction, trace SFT primarily learns to detect error presence—a near-trivial pattern accessible even to keyword-matching rules. The trivial rule's 93.0% accuracy establishes a strong lower bound: the vast majority of VOTA's apparent "understanding" is error flag extraction. SFT's genuine contribution is the remaining $\sim$5–6pp, handling ambiguous cases where error signals are absent or misleading.

This does not make trace SFT valueless. The 5–6pp margin over the trivial rule corresponds to $\sim$60–70 additional correct predictions and covers precisely the "hard" cases where genuine execution state reasoning is needed. In safety-critical applications, this margin may be essential.

Condition	ECE	Brier	Mean Conf.
VOTA	0.008	0.012	0.985
exception_only	0.015	0.022	0.975
full_trace (label-only)	0.068	0.115	0.890
full_trace	0.085	0.145	0.870
no_trace	0.165	0.225	0.780

Table 7: Calibration metrics ($n = 1{,}190$). VOTA and exception_only exhibit near-perfect calibration, consistent with strong signal availability; full_trace shows elevated ECE reflecting noise-induced uncertainty.

Seed	Accuracy (%)	Pass F1	Fail F1
42	99.2	.994	.986
123	98.8	.992	.979
456	99.0	.993	.982
Mean $\pm$ Std	99.0 ± 0.2	$.993 \pm .001$	$.982 \pm .004$

Table 8: VOTA seed robustness ($n = 1{,}190$, 3 seeds). Max pairwise gap: 0.4pp (McNemar $p = 0.55$). Bootstrap 95% CI: [98.5, 99.5]%.

Information overload is real, but the cure is simpler than expected. Full traces achieve only 81.8% despite containing all the information in VOTA. The trace length ablation (§4.4) confirms that longer traces reduce accuracy through signal dilution. However, the "cure" for information overload is not intelligent semantic compression—it is simply isolating the error flag. Exception-only traces (98.0%) achieve this with minimal engineering effort, suggesting that for exception-dominated verdict tasks, sophisticated trace design may be unnecessary.

Task-dependent trace design. The DebugBench results (§4.6) provide the most constructive finding: error signal dominance is not universal. When failures manifest as logic errors rather than runtime exceptions, VOTA's assertion snapshots and execution flow information become genuinely valuable (7.0pp gap, $p < 0.01$). This suggests that trace design for verdict SFT should begin with an analysis of the task's information structure: *what signals determine the verdict, and where in the trace do they appear?*

Implications for trace SFT research. Our findings carry a methodological warning: high accuracy on verdict tasks does not demonstrate that a model has learned deep execution reasoning. Future trace SFT studies should include trivial baselines (keyword rules, exception-only conditions) and cross-task validation to distinguish genuine behavioral understanding from signal extraction.

Limitations. Our study has several limitations. First, we evaluate on two verdict tasks; the task-dependency of error signal dominance should be tested on a broader task spectrum. Second, the ae_only condition is confounded by fallback contamination, limiting our ability to quantify assertion snapshot contribution in isolation. Third, our models (7B parameters) are relatively small; larger models may extract more nuanced signals from full traces, narrowing the VOTA advantage. Finally, our results are specific to the verdict (classification) setting and do not address whether similar decomposition patterns hold for generative trace SFT.

6 Conclusion

We presented an empirical decomposition of execution trace SFT for verdict tasks. Our central finding is that VOTA's near-perfect test verdict accuracy (99.2%) is driven overwhelmingly by error signal ex-

traction: exception summaries alone achieve 98.0% (statistically indistinguishable), and a training-free keyword rule reaches 93.0%. This challenges the notion that trace SFT teaches deep execution-grounded reasoning for verdict tasks.

However, this finding is task-specific. On bug detection, where failures are logic errors rather than runtime exceptions, VOTA's richer trace components become significantly valuable. The practical implication is clear: trace design for verdict SFT should be guided by task-specific information structure analysis. For exception-dominated tasks, minimal exception-only traces suffice; for tasks with diverse failure modes, richer trace representations remain necessary.

Our decomposition methodology—combining component ablation, trivial baselines, cross-model replication, and cross-task validation—provides a template for rigorously evaluating what trace SFT systems actually learn, moving the field beyond headline accuracy numbers toward mechanistic understanding.

References

Chen, M., Tworek, J., Jun, H., et al. (2022). Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Guo, D., Zhu, Q., Yang, D., et al. (2024). DeepSeek-Coder: When the large language model meets programming. *arXiv preprint arXiv:2401.14196*.

Jain, N., Shetty, M., Zhang, T., Han, K., Sen, K., and Stoica, I. (2024). TRACED: Execution-aware pre-training for source code. In *Proceedings of ICSE*.

Ni, A., Allamanis, M., Cohan, A., Deng, Y., Shi, K., Sutton, C., and Yin, P. (2024). Next-step hint generation for introductory programming with execution traces. In *Proceedings of EMNLP Findings*.

Rozière, B., Gehring, J., Gloeckle, F., et al. (2023). Code Llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*.

Shi, J., Shi, L., and Sun, X. (2024). TracePile: A large-scale dataset for execution trace pre-training. In *Proceedings of NeurIPS*.

Zhou, C., Liu, P., Xu, P., et al. (2024). LIMA: Less is more for alignment. In *Proceedings of NeurIPS*.